Deep Learning Facial Recognition

Turning a Face into Numbers with Encodings and Triplet Loss

A standalone teaching guide for beginners. It explains how a deep-learning system recognizes faces, and walks through the companion script `deep_learning_facial_recognition.py`, a small but complete demo you can run yourself.

THREE THINGS TO REMEMBER

1) Face VERIFICATION solves an easier 1:1 matching problem; face RECOGNITION solves a harder 1:K matching problem. 2) The TRIPLET LOSS is an effective loss function for training a network to learn an encoding of a face image. 3) The SAME encoding works for both tasks - measuring the distance between two images' encodings tells you whether they show the same person.

1. The Core Idea: A Face Becomes Numbers

Comparing two face photos pixel by pixel does not work - the same person looks different under new lighting, angles, or expressions, and two different people can have similar pixels. Deep-learning face recognition sidesteps this entirely. A neural network converts each face into a short list of numbers called an ENCODING (also called an embedding). In the model used here, that list has 128 numbers.

The network is trained so the encoding captures the IDENTITY of the face, not the lighting or pose. The payoff is simple: once each face is a vector of numbers, comparing faces just means measuring the DISTANCE between two vectors.

> **DISTANCE IS EVERYTHING**
> Two photos of the SAME person produce encodings that are close together (small distance). Photos of DIFFERENT people produce encodings that are far apart (large distance). A threshold distance decides 'same' versus 'different'.

For more background on what an embedding is, see Google's beginner-friendly overview: ML Crash Course: Embeddings.

2. Two Tasks: Verification vs. Recognition

These two words sound similar but describe different problems, and one is much harder than the other.

	Verification (1:1)	Recognition (1:K)
Question	'Are you who you claim to be?'	'Who are you?'
Input	A face PLUS a claimed name	A face only
Compared against	One stored encoding	All K stored encodings
Difficulty	Easier	Harder

Verification is a single yes/no comparison: encode the new face, look up the claimed person's stored encoding, and check whether the distance is small. Recognition has no name to start from, so it must compare the new face against every person in the database and pick the closest - with more people, there are more chances for a wrong match, which is why 1:K is harder than 1:1.

3. How the Encoder Learns: Triplet Loss

How do you train a network to produce good encodings? You show it TRIPLETS of images. Each triplet has three parts:

- **Anchor (A)** - a reference photo of some person.
- **Positive (P)** - a different photo of the SAME person.
- **Negative (N)** - a photo of a DIFFERENT person.

The triplet loss pushes the anchor's encoding CLOSER to the positive and FARTHER from the negative, with a safety gap called the margin (alpha):

```
pos_dist = sum( (f(A) - f(P))^2 )      # want this SMALL
neg_dist = sum( (f(A) - f(N))^2 )      # want this LARGE
loss     = max( pos_dist - neg_dist + alpha , 0 )
```

WHY THE MARGIN ALPHA MATTERS

Without the margin, the network could cheat by mapping every face to the same point, making all distances zero. Alpha forces a real gap between same-person and different-person distances, so the encoding has to be meaningful.

This is the loss introduced in the FaceNet paper, the main work behind this approach: FaceNet: A Unified Embedding for Face Recognition and Clustering.

4. Walking Through the Code

4.1 Encoding a face

Every operation starts by turning an image into its 128-number encoding. In a real system this calls a pretrained FaceNet model; the companion script uses a small stand-in encoder so it runs instantly with no downloads, but the surrounding logic is identical.

4.2 Verification (1:1)

Encode the live photo, fetch the claimed person's stored encoding, measure the distance, and open the door only if it is below the threshold:

```
encoding = img_to_encoding(image, model)
dist = np.linalg.norm(encoding - database[identity])
if dist < threshold:
    door_open = True      # same person
else:
    door_open = False     # imposter
```

4.3 Recognition (1:K)

With no name to look up, loop over the whole database and keep the closest match. If even the closest is too far, report the person as unknown:

```
min_dist = 100
for (name, db_enc) in database.items():
    dist = np.linalg.norm(encoding - db_enc)
    if dist < min_dist:
        min_dist = dist
        identity = name
```

ONE ENCODING, TWO TASKS

Notice that 4.2 and 4.3 use the exact same img_to_encoding() and the exact same distance measurement. Only the comparison logic differs - one name versus the whole database. That is the point: a single learned encoding serves both verification and recognition.

5. How to Run It, and What You Will See

Install the requirements and run:

```
python -m pip install numpy tensorflow
python deep_learning_facial_recognition.py
```

The script first prints the triplet-loss value on a test triplet (the quantity a real encoder minimizes during training), then enrolls three known people and runs two verification demos and two recognition demos. The output looks like this:

```
[Triplet loss] value on a random test triplet:
  loss = 527.259827

[Database] enrolling three known employees: danielle, younes, kian

[Verification - 1:1] 'I am younes' (a fresh photo of younes):
  It's younes, welcome in!   (distance 0.0024)

[Verification - 1:1] 'I am kian' (but it is really a stranger):
  It's not kian, please go away.   (distance 0.5802)

[Recognition - 1:K] who is this? (a fresh photo of younes):
  It's younes, the distance is 0.0024

[Recognition - 1:K] who is this? (a complete stranger):
  Not in the database.   (closest was younes at 0.4413)
```

How to read those results:

- **younes is accepted** in both demos with a tiny distance (~0.002), because the fresh photo's encoding sits almost on top of his stored encoding.
- **The stranger is rejected** in both demos: verification says 'please go away' and recognition says 'Not in the database', because the smallest distance found (~0.44-0.58) is larger than the threshold.
- **The same encoding drove both** - verification compared against one person, recognition against everyone.

> **A NOTE ON THE THRESHOLD**
> The cutoff distance depends on the encoder's scale. The real FaceNet assignment uses a threshold of 0.7; this demo's simpler stand-in encoder separates people at much smaller distances, so it uses 0.1. The principle is the same: below the threshold means 'same person'. Real systems tune this number on real data.

5.1 Wait - where did the 'photo' go?

A beginner reading the output might expect the program to be holding actual pictures and comparing them. It is not. The moment any image enters the system, it is immediately passed through `img_to_encoding()` and becomes a list of 128 numbers. From that point on, the 'photo' only exists as that list - the ENCODING. Every distance you see in the output (0.0024, 0.5802, 0.4413) is the distance between two of these number-lists, NOT between two images.

THE 'PHOTO' IS REALLY A LIST OF NUMBERS

When the output says 'a fresh photo of younes', what the code actually compares is younes's NEW 128-number encoding against his STORED 128-number encoding. The database never holds pictures at all - only encodings. So 'distance 0.0024' means: younes's two encodings differ by only 0.0024, therefore it is the same person. The image was converted to numbers first; the comparison is always number-to-number.

Why not just compare the two images pixel by pixel? Because that fails badly:

- **Same person, different pixels.** A new photo of younes has different lighting, angle, and expression, so thousands of his pixels change - even though it is still him. Pixel-by-pixel distance would call the two photos 'different'.
- **Different people, similar pixels.** Two different people photographed against the same wall in the same lighting can share many pixel values, fooling a pixel comparison into calling them 'the same'.
- **Pixels describe appearance, not identity.** The encoder is trained (via the triplet loss) to throw away lighting and pose and keep only what makes a face that PERSON. Its 128 numbers capture identity, which raw pixels never do.

That is the whole reason the encoding step exists: it converts an unreliable pile of pixels into a compact, identity-focused list of numbers that distances can be trusted on.

6. Quick Glossary

Term	Meaning
Encoding / embedding	The list of numbers (here 128) representing a face.
Verification (1:1)	Checking a face against ONE claimed identity.
Recognition (1:K)	Identifying a face among K known people.
Triplet	An (Anchor, Positive, Negative) set of training images.
Triplet loss	Loss that pulls A-P close and pushes A-N apart.
Margin (alpha)	Required gap between same- and different-person distances.
L2 distance	Straight-line distance between two encodings.
Threshold	Cutoff distance for deciding 'same' vs 'different'.

7. Learn More (Beginner-Friendly Links)

- Embeddings - Google ML Crash Course - what an encoding/embedding actually is.
- An Introduction to Convolutional Neural Networks - CNN basics, the kind of network that encodes faces.
- Going Deeper with Convolutions (Inception) - the Inception architecture referenced by the model.
- NumPy linalg.norm - the function used to measure distance between encodings.
- TensorFlow tf.data guide - how image input pipelines are built.
- Keras image loading & preprocessing - loading and resizing face images.
- Deep Learning and Face Recognition: The State of the Art - broader background after FaceNet.

8. References

1. Florian Schroff, Dmitry Kalenichenko, James Philbin (2015). FaceNet: A Unified Embedding for Face Recognition and Clustering.
2. Yaniv Taigman, Ming Yang, Marc'Aurelio Ranzato, Lior Wolf (2014). DeepFace: Closing the Gap to Human-Level Performance in Face Verification.
3. The official FaceNet GitHub repository: github.com/davidsandberg/facenet.
4. Machine Learning Mastery: How to Develop a Face Recognition System Using FaceNet in Keras.
5. keras-facenet conversion notebook: github.com/nyoki-mtl/keras-facenet.